Mini Voice Interface

modifies history

Version	Contributor	Date	Change Log
v1.0	logic.peng	18/08/10	init

Summary

Mini Sound Management Service is a service application in Robot that manages the playback control of voices at different priorities in Mini. This document briefly describes these interfaces and SDK integration

SDK

- protobuf struct

```
//MiniMediaPlayer voice source type
enum Source {
    RING = 0;//ring
    ALARM = 1;//alarm
    SPEECH = 2; //speech tts
    MUSIC = 3; //music
    BEHAVIOR = 4; //behavior dubbing
}
```

- TTS Interface

```
public class VoicePool {
/**
  * TTs
  *
  * @param text the text of TTs
  * @param priority  {@link Priority}
  * @param listener tts state callback
  */
 public void playTTs(String text, Priority priority, final VoiceListener listener);

 /**
  * Play local TTs audio files
  *
  * @param mp3FileName /sdcard/localTTs/mp3FileName.mp3
  * @param priority  {@link Priority}
  * @param listener tts state callback
  */
 public void playLocalTTs(String mp3FileName, Priority priority, final VoiceListener listener) '

 /**
  * stop TTS
  *
  * @param priority  {@link Priority}
  * @param listener tts ResponseListener callback
  */
 public void stopTTs(Priority priority, @Nullable final ResponseListener<Void> listener) ;

 /**
  * Play local TTs audio files and do not participate in VoiceManage's voice conflict management
  *
  * @param mp3FileName /sdcard/localTTs/mp3FileName.mp3
  * @param listener tts state callback
  */
 public void playUnsafeTTs(String mp3FileName, @Nullable final VoiceListener listener);
```

```
}

/**
 * voice listener
 */

public interface VoiceListener {
  /**
   * completed listener
   */
  void onCompleted();

  /**
   * error listener
   *
   * @param errCode 0 indicates success, others express mistakes, {@link VoiceCode}
   * @param errMsg error message
   */
  void onError(int errCode, String errMsg);
}

/**
 * Voice Error Code
 */
public final class VoiceCode {
  public static final int MEDIA_NOT_EXIST = 100;
  public static final int SESSION_CLOSED = 101;
  public static final int MEDIA_FORBIDDEN = 102;
  public static final int INVALID_PARAM = 103;
  public static final int BAD_RESPOND = 104;
}
```

- MiniMediaPlayer

```
public final class MiniMediaPlayer {

 public static MiniMediaPlayer create(MasterContext context, VoiceProto.Source source)
    throws VoiceException {
  return new MiniMediaPlayer(context, source);
 }

 /**
  * Sets the data source as a content Uri.
  *
  * @param path the Content URI of the data you want to play
  * @throws IOException if it is called in an invalid state
  * @throws IllegalArgumentException path argument
  * @throws SecurityException security
  * @throws IllegalStateException illegal state
  */
 public void setDataSource(String path)
    throws IOException, IllegalArgumentException, SecurityException, IllegalStateException ;

 /**
  * Prepares the player for playback, synchronously.
  * <p>
  * After setting the datasource and the display surface, you need to either
  * call prepare() or prepareAsync(). For files, it is OK to call prepare(),
  * which blocks until MediaPlayer is ready for playback.
  *
  * @throws IllegalStateException if it is called in an invalid state
  */
 public void prepare() throws IOException, IllegalStateException ;

 /**
  * Prepares the player for playback, synchronously.
  * <p>
  * After setting the datasource and the display surface, you need to either
  * call prepare() or prepareAsync(). For files, it is OK to call prepare(),
```

```
     * which blocks until MediaPlayer is ready for playback.
     *
     * @throws IllegalStateException if it is called in an invalid state
     */
    public void prepareAsync() throws IllegalStateException ;

    /**
     * Starts or resumes playback. If playback had previously been paused,
     * playback will continue from where it was paused. If playback had
     * been stopped, or never started before, playback will start at the
     * beginning. if playback source is low priority than other playback,
     * be rejected, return false.
     */
    public void start();

    /**
     * Pauses playback. Call start() to resume.
     *
     * @throws IllegalStateException if the internal player engine has not been
     * initialized.
     */
    public void pause() throws IllegalStateException {
      mediaPlayer.pause();
      faker.pause();
    }

    /**
     * Stops playback after playback has been started or paused.
     *
     * @throws IllegalStateException if the internal player engine has not been
     * initialized.
     */
    public void stop() ;

    /**
     * Resets the MediaPlayer to its uninitialized state. After calling
     * this method, you will have to initialize it again by setting the
```

```
     * data source and calling prepare().
     */
    public void reset() ;

    /**
     * Releases resources associated with this MediaPlayer object.
     * It is considered good practice to call this method when you're
     * done using the MediaPlayer. In particular, whenever an Activity
     * of an application is paused (its onPause() method is called),
     * or stopped (its onStop() method is called), this method should be
     * invoked to release the MediaPlayer object, unless the application
     * has a special need to keep the object around. In addition to
     * unnecessary resources (such as memory and instances of codecs)
     * being held, failure to call this method immediately if a
     * MediaPlayer object is no longer needed may also lead to
     * continuous battery consumption for mobile devices, and playback
     * failure for other applications if no multiple instances of the
     * same codec are supported on a device. Even if multiple instances
     * of the same codec are supported, some performance degradation
     * may be expected when unnecessary multiple instances are used
     * at the same time.
     */
    public void release() ;

    /**
     * Seeks to specified time position.
     *
     * @param msec the offset in milliseconds from the start to seek to
     * @throws IllegalStateException if the internal player engine has not been
     * initialized
     */
    public void seekTo(int msec) ;

    /**
     * Checks whether the MediaPlayer is playing.
     *
     * @return true if currently playing, false otherwise
```

```
     * @throws IllegalStateException if the internal player engine has not been
     * initialized or has been released.
     */
    public boolean isPlaying() ;

    /**
     * Sets the volume on this player.
     * This API is recommended for balancing the output of audio streams
     * within an application. Unless you are writing an application to
     * control user settings, this API should be used in preference to
     * {@link AudioManager#setStreamVolume(int, int, int)} which sets the volume of ALL streams of
     * a particular type. Note that the passed volume values are raw scalars in range 0.0 to 1.0.
     * UI controls should be scaled logarithmically.
     *
     * @param volume left volume scalar
     */
    public void setVolume(float volume) ;

    /**
     * Gets the duration of the file.
     *
     * @return the duration in milliseconds, if no duration is available
     * (for example, if streaming live content), -1 is returned.
     */
    public int getDuration() ;

    /**
     * Gets the current playback position.
     *
     * @return the current position in milliseconds
     */
    public int getCurrentPosition() ;

    /**
     * Register a callback to be invoked when the media source is ready
     * for playback.
     *
```

```
     * @param listener the callback that will be run
     */
    public void setOnPreparedListener(@Nullable MiniMediaPlayer.OnPreparedListener listener) ;
    /**
     * Register a callback to be invoked when the end of a media source
     * has been reached during playback.
     *
     * @param listener the callback that will be run
     */
    public void setOnCompletionListener(@Nullable MiniMediaPlayer.OnCompletionListener listener) ;

    /**
     * Register a callback to be invoked when the status of a network
     * stream's buffer has changed.
     *
     * @param listener the callback that will be run.
     */
    public void setOnBufferingUpdateListener(
        @Nullable MiniMediaPlayer.OnBufferingUpdateListener listener) ;

    /**
     * Register a callback to be invoked when a seek operation has been
     * completed.
     *
     * @param listener the callback that will be run
     */
    public void setOnSeekCompleteListener(@Nullable MiniMediaPlayer.OnSeekCompleteListener listener) ;

    /**
     * Register a callback to be invoked when an error has happened
     * during an asynchronous operation.
     *
     * @param listener the callback that will be run
     */
    public void setOnErrorListener(@Nullable MiniMediaPlayer.OnErrorListener listener) ;

    public interface OnErrorListener {
```

```
    boolean onError(MiniMediaPlayer mp, int what, int extra);
  }

  public interface OnSeekCompleteListener {
    void onSeekComplete(MiniMediaPlayer mp);
  }

  public interface OnBufferingUpdateListener {
    void onBufferingUpdate(MiniMediaPlayer mp, int percent);
  }

  public interface OnCompletionListener {
    void onCompletion(MiniMediaPlayer mp);
  }

  public interface OnPreparedListener {
    void onPrepared(MiniMediaPlayer mp);
  }
}
```